

EN3160 Assignment 1 on Intensity Transformations and Neighb

Index : 230544C

Name : Rathnayake M. A. G. K. N.

GitHub link to the folder: <https://github.com/codingNR29/Image-processing/tree/5b7cef611a8a2a0e6ab166633ee592c571c799fe/Assignments/Assignment%201>

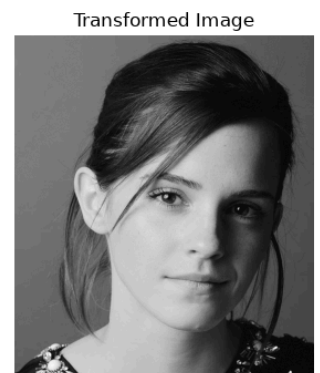
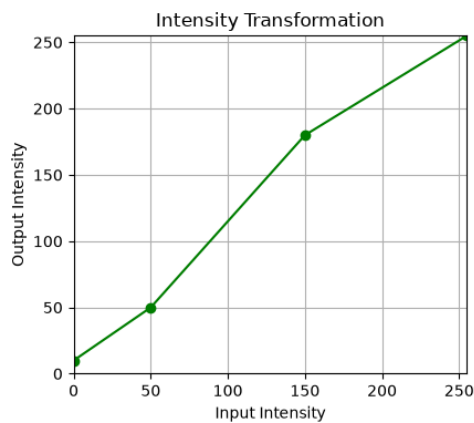
GitHub link to the repo: <https://github.com/codingNR29/Image-processing.git>

```
1 #Importing the libraries
2 import numpy as np
3 import cv2 as cv
4 import matplotlib.pyplot as plt
```

Q1)

```
1 #Defining the function for the intensity transformation
2 def intensity_transform(im, breakpoints):
3
4     #Selecting the input and output intensities from the breakpoints
5     x = breakpoints[:, 0] #Input values
6     y = breakpoints[:, 1] #Output values
7
8     #Matching the breakpoints and creating a function
9     tranformation = np.interp(im, x, y)
10
11     return tranformation
12
13 #Importin the initial image
14 im_orig = cv.imread('Emma.jpg', cv.IMREAD_GRAYSCALE)
15
16 #Applying transformation
17 breakpoints = np.array([[0, 0], [50, 50], [50, 100], [150, 255], [150, 150], [255, 255]])
18 im_transformed = intensity_transform(im_orig, breakpoints)
```

```
1 #Trying different breakpoints to get a visually pleasing output
2 breakpoints = np.array([[0, 10], [50, 50], [150, 180], [255, 255]])
3 im_transformed = intensity_transform(im_orig, breakpoints)
4
5 #Plotting
6 fig, ax = plt.subplots(1, 3, figsize= (12, 4))
7
8 #Transformation function
9 ax[0].plot(breakpoints[:, 0], breakpoints[:, 1], marker='o', color= 'green')
10 ax[0].set_title('Intensity Transformation')
11 ax[0].set_xlabel('Input Intensity')
12 ax[0].set_ylabel('Output Intensity')
13 ax[0].set_xlim(0, 255)
14 ax[0].set_ylim(0, 255)
15 ax[0].grid()
16
17 #Original image
18 ax[1].imshow(im_orig, cmap = 'gray', vmin = 0, vmax = 255)
19 ax[1].set_title('Original Image')
20 ax[1].axis('off')
21
22 #Transformed image
23 ax[2].imshow(im_transformed, cmap = 'gray', vmin = 0, vmax = 255)
24 ax[2].set_title('Transformed Image')
25 ax[2].axis('off')
26
27 plt.tight_layout()
28 plt.show()
29
```



The intensity transformation changes selected intensity ranges to improve the visual contrast of the image. The selected breakpoints produced a clearer and more visually pleasing output.

Q2)

```
1 #Importing brain image
2 im_brain = cv.imread('Brain.png', cv.IMREAD_GRAYSCALE)
3
4 #Caluclating histogram
5 hist = cv.calcHist([im_brain], [0], None, [256], [0,256])
```

```
1 fig, ax = plt.subplots(1, 3, figsize=(12, 4))
2
3 # Different candidate intensity ranges
4 range1 = ((im_brain >= 150) & (im_brain < 175))
5 range2 = ((im_brain >= 175) & (im_brain < 200))
6 range3 = ((im_brain >= 200) & (im_brain < 230))
7
8 ax[0].imshow(range1, cmap='gray')
9 ax[0].set_title('150 - 174')
10
11 ax[1].imshow(range2, cmap='gray')
12 ax[1].set_title('175 - 199')
13
14 ax[2].imshow(range3, cmap='gray')
15 ax[2].set_title('200 - 229')
16
17 for a in ax:
18     a.axis('off')
19
20 plt.show()
```

150 - 174



175 - 199



200 - 229



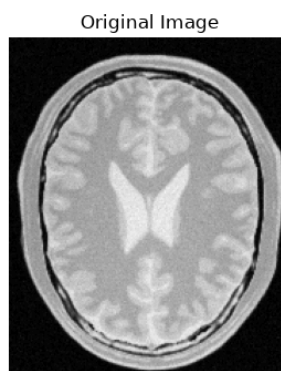
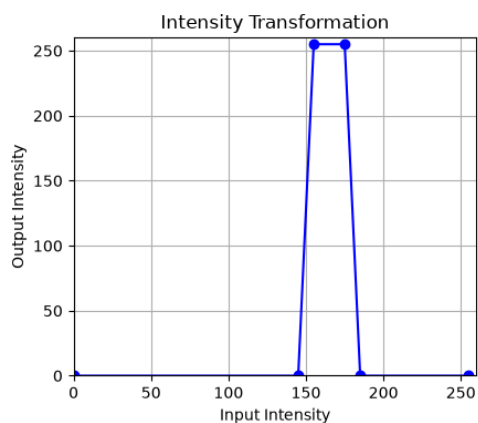
Q2) a)

```
1 #Creating breakpoints for the white matter attenuation
2 breakpoints_white_matter_attenuation = np.array([[0, 0], [145, 0], [155, 255], [175, 255], [185, 0], [255, 0]])
```

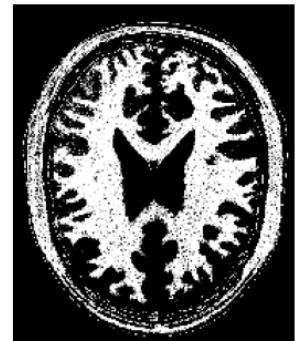
```

3
4 im_white_matter_accenuated = intensity_transform(im_brain, breakpoints_white_matter_accenuation)
5
6 #Plotting
7 fig, ax = plt.subplots(1, 3, figsize= (12, 4))
8
9 #Transformation function
10 ax[0].plot(breakpoints_white_matter_accenuation[:, 0], breakpoints_white_matter_accenuation[:, 1], marker='o', color=
11 ax[0].set_title('Intensity Transformation')
12 ax[0].set_xlabel('Input Intensity')
13 ax[0].set_ylabel('Output Intensity')
14 ax[0].set_xlim(0, 260)
15 ax[0].set_ylim(0, 260)
16 ax[0].grid()
17
18 #Original image
19 ax[1].imshow(im_brain, cmap = 'gray', vmin = 0, vmax = 255)
20 ax[1].set_title('Original Image')
21 ax[1].axis('off')
22
23 #Transformed image
24 ax[2].imshow(im_white_matter_accenuated, cmap = 'gray', vmin = 0, vmax = 255)
25 ax[2].set_title('White matter accenuated image')
26 ax[2].axis('off')
27
28 plt.tight_layout()
29 plt.show()
30

```



White matter accenuated image



The selected intensity transformation increases the contrast of the white matter region.
Therefore, the white matter becomes more clearly visible compared with the surrounding regions.

Q2) b)

```

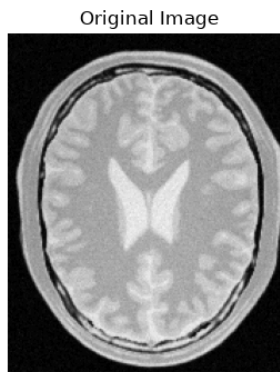
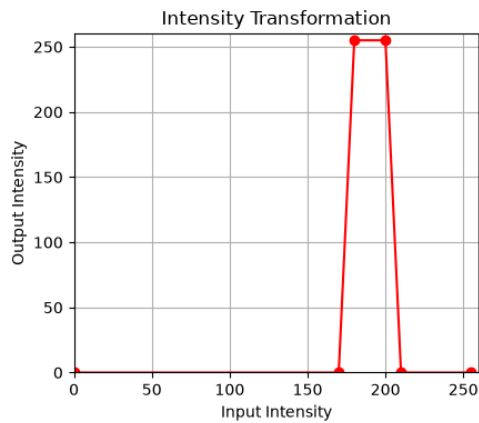
1 #Creating breakpoints for the grey matter attenuation
2 breakpoints_grey_matter_accenuation = np.array([[0, 0], [170, 0], [180, 255], [200, 255], [210, 0], [255, 0]])
3
4 im_grey_matter_accenuated = intensity_transform(im_brain, breakpoints_grey_matter_accenuation)
5
6 #Plotting
7 fig, ax = plt.subplots(1, 3, figsize= (12, 4))
8
9 #Transformation function
10 ax[0].plot(breakpoints_grey_matter_accenuation[:, 0], breakpoints_grey_matter_accenuation[:, 1], marker='o', color=
11 ax[0].set_title('Intensity Transformation')
12 ax[0].set_xlabel('Input Intensity')
13 ax[0].set_ylabel('Output Intensity')
14 ax[0].set_xlim(0, 260)
15 ax[0].set_ylim(0, 260)
16 ax[0].grid()
17
18 #Original image
19 ax[1].imshow(im_brain, cmap = 'gray', vmin = 0, vmax = 255)
20 ax[1].set_title('Original Image')
21 ax[1].axis('off')
22
23 #Transformed image
24 ax[2].imshow(im_grey_matter_accenuated, cmap = 'gray', vmin = 0, vmax = 255)
25 ax[2].set_title('Grey matter accenuated image')

```

```

26 ax[2].axis('off')
27
28 plt.tight_layout()
29 plt.show()

```



Grey matter accentuated image



The selected intensity transformation enhances the intensity range corresponding to gray matter. This makes the gray matter easier to distinguish from the other regions.

Q3) a)

```

1 #Importing the image
2 im_orig = cv.imread('Legs.jpg')
3
4 #Converting the image to L*a*b*
5 im_lab = cv.cvtColor(im_orig, cv.COLOR_BGR2LAB)
6
7 #Splitting the three channels
8 L, a, b = cv.split(im_lab)
9
10 # Gamma correction lookup table
11 gamma = 0.7
12
13 table = np.array([(i / 255.0) ** gamma * 255.0 for i in np.arange(256)]).astype(np.uint8)
14
15 #Applying gamma correction to the L channel
16 L_gamma = cv.LUT(L, table)
17
18 #Merging back
19 im_lab_L_gamma = cv.merge([L_gamma, a, b])
20
21 #Converting the colour space back to BGR
22 im_L_gamma = cv.cvtColor(im_lab_L_gamma, cv.COLOR_LAB2BGR)
23
24 #To display using matplotlib these need to be converted into RGB
25 im_rgb = cv.cvtColor(im_orig, cv.COLOR_BGR2RGB)
26 im_L_gamma_rgb = cv.cvtColor(im_L_gamma, cv.COLOR_BGR2RGB)
27
28 #Printing the gamma value
29 print("The used gamma value is", gamma)
30
31 #Display
32 fig, ax = plt.subplots(1, 2, figsize = (10, 5))
33
34 ax[0].imshow(im_rgb)
35 ax[0].set_title('Original Image')
36
37 ax[1].imshow(im_L_gamma_rgb)
38 ax[1].set_title('Image with gamma corrected L plane')
39
40 for a in ax:
41     a.axis('off')
42
43 plt.show()

```

The used gamma value is 0.7

Original Image



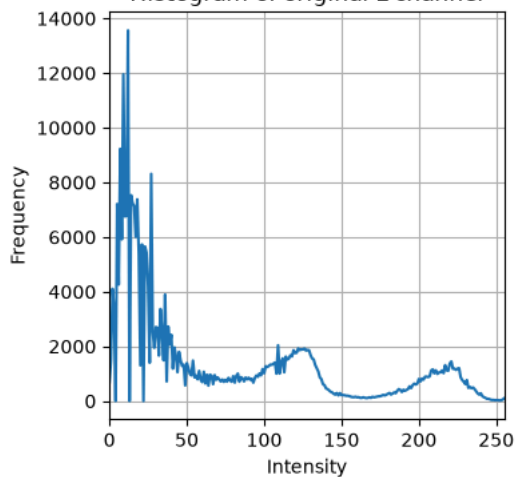
Image with gamma correcte L plane



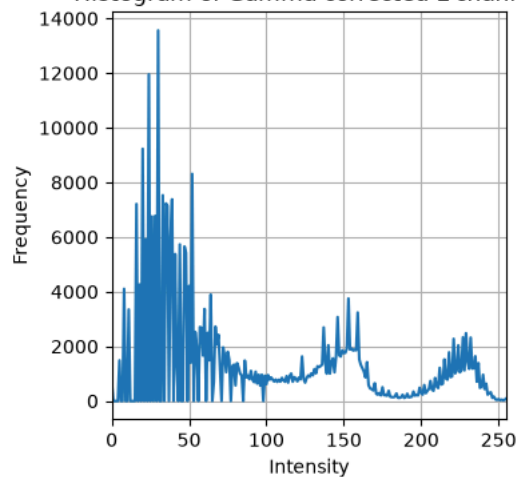
Q3) b)

```
1 # Histograms of L channel before and after gamma correction
2 hist_L_orig = cv.calcHist([L], [0], None, [256], [0, 256])
3 hist_L_gamma = cv.calcHist([L_gamma], [0], None, [256], [0, 256])
4
5 fig, ax = plt.subplots(1, 2, figsize=(8, 4))
6
7 ax[0].plot(hist_L_orig)
8 ax[0].set_title('Histogram of original L channel')
9 ax[0].set_xlabel('Intensity')
10 ax[0].set_ylabel('Frequency')
11 ax[0].set_xlim(0, 255)
12 ax[0].grid()
13
14 ax[1].plot(hist_L_gamma)
15 ax[1].set_title('Histogram of Gamma corrected L channel')
16 ax[1].set_xlabel('Intensity')
17 ax[1].set_ylabel('Frequency')
18 ax[1].set_xlim(0, 255)
19 ax[1].grid()
20
21 plt.tight_layout()
22 plt.show()
```

Histogram of original L channel



Histogram of Gamma corrected L channel



Q4) a)

```
1 #Importing the image
2 im_4 = cv.imread('SpiderMan.jpg')
3
4 #Converting into HSV planes
5 im_4_HSV = cv.cvtColor(im_4, cv.COLOR_BGR2HSV)
6
7 #Splitting the channels
8 H, S, V = cv.split(im_4_HSV)
```

Q4) b)

```
1 # Applying the transformation
2 a = 0.5
```

```

3 sigma = 70
4
5 #Input saturation intensities
6 x = np.arange(256, dtype=np.float32)
7
8 #Vibrance transformation
9 y = x + a * 128 * np.exp(-((x - 128)**2) / (2 * sigma**2))
10
11 #Limiting the output to 255
12 y = np.minimum(y, 255)
13
14 #Creating the lookup table
15 table = y.astype(np.uint8)
16
17 #Applying transformation to the saturation plane
18 new_S = cv.LUT(S, table)

```

Q4) c)

```
1 print("Visually pleasing a value is",a)
```

Visually pleasing a value is 0.5

Q4) d)

```

1 #Combining the 3 channels to see the final image again
2 im_4_new = cv.merge([H, new_S, V])
3
4 #BGR image
5 im_4_new_BGR = cv.cvtColor(im_4_new, cv.COLOR_HSV2BGR)
6
7 #RGB image
8 im_4_new_RGB = cv.cvtColor(im_4_new_BGR, cv.COLOR_BGR2RGB)

```

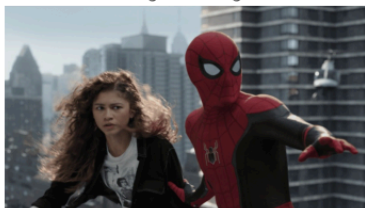
Q4) e)

```

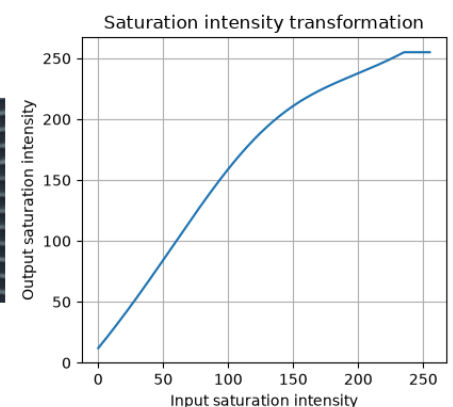
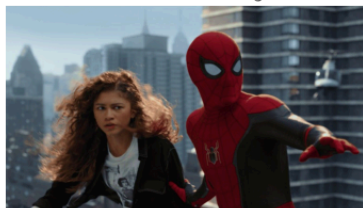
1 fig, ax = plt.subplots(1, 3, figsize= (12, 4))
2
3 ax[0].imshow(cv.cvtColor(im_4, cv.COLOR_BGR2RGB))
4 ax[0].set_title('Original Image')
5 ax[0].axis('off')
6
7 ax[1].imshow(im_4_new_RGB)
8 ax[1].set_title('Transformed Image')
9 ax[1].axis('off')
10
11 ax[2].plot(x, y)
12 ax[2].set_xlabel('Input saturation intensity')
13 ax[2].set_ylabel('Output saturation intensity')
14 ax[2].set_title('Saturation intensity transformation')
15 ax[2].grid()
16
17 plt.tight_layout()
18 plt.show()

```

Original Image



Transformed Image



The saturation plane was modified while the hue and value planes were kept unchanged.
This increases the strength of the colors and produces a more vibrant image.

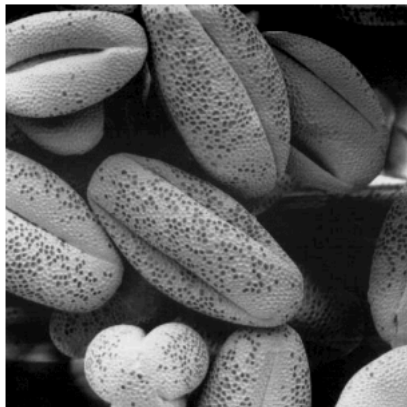
Q5)

```

1 #Importing the image
2 im_5 = cv.imread('SeaShells.png', cv.IMREAD_GRAYSCALE)
3
4 #Calculating the histogram
5 im_5_hist,im_5_bins = np.histogram(im_5.ravel(),256,[0,256])
6
7 #Funtion for histogram equilization
8 def hist_eq(image):
9
10     #Calculating histogram
11     hist,bins = np.histogram(image.ravel(),256,[0,256])
12
13     #Normalizing
14     normalized_hist = hist/ image.size
15
16     #Cumulative distribution function
17     cdf = np.cumsum(normalized_hist)
18
19     #Tranformation
20     tranform = np.round(255 * cdf).astype(np.uint8)
21
22     #Euqilizing the image
23     eq_image = tranform[image]
24
25     return eq_image
26
27 #Applying the function
28 im_5_hist_eqd = hist_eq(im_5)
29
30 #Calulating the new histogram
31 im_5_hist_eqd_hist,im_5_hist_eqd_bins = np.histogram(im_5_hist_eqd.ravel(),256,[0,256])
32
33 fig, ax = plt.subplots(2, 2, figsize=(8, 8))
34
35 ax[0, 0].imshow(im_5, cmap='grey')
36 ax[0, 0].set_title('Original image')
37 ax[0, 0].axis('off')
38
39 ax[0, 1].imshow(im_5_hist_eqd, cmap='grey')
40 ax[0, 1].set_title('Histogram equalized image')
41 ax[0, 1].axis('off')
42
43 ax[1, 0].plot(im_5_hist, color='green')
44 ax[1, 0].set_title('Histogram of the original image')
45 ax[1, 0].set_xlabel('Intensity value')
46 ax[1, 0].set_ylabel('Number of pixels')
47 ax[1, 0].grid()
48
49 ax[1, 1].plot(im_5_hist_eqd_hist)
50 ax[1, 1].set_title('Histogram of the histogram equalized image')
51 ax[1, 1].set_xlabel('Intensity value')
52 ax[1, 1].set_ylabel('Number of pixels')
53 ax[1, 1].grid()
54
55 plt.tight_layout()
56 plt.show()

```

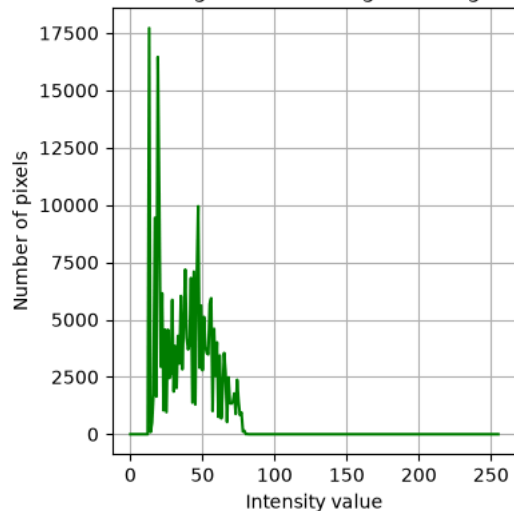
Original image



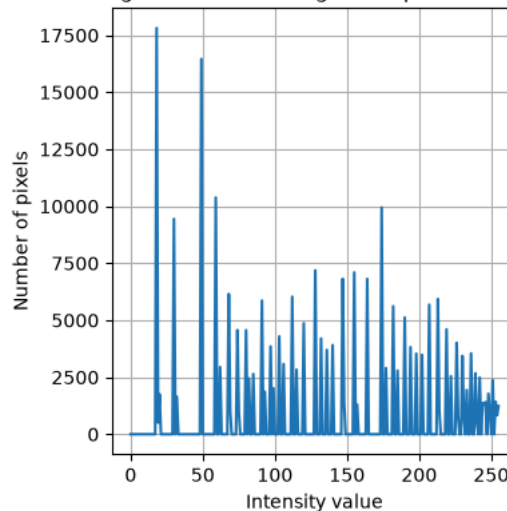
Histogram equalized image



Histogram of the original image



Histogram of the histogram equalized image



Histogram equalization spreads the intensity values over a wider range. Therefore, the contrast of the image is improved and more details become visible.

Q6) a)

```
1 #Importing the image
2 im_6 = cv.imread('Jennifer.png')
3
4 #Separating the H, S, V planes
5 H, S, V = cv.split(cv.cvtColor(im_6, cv.COLOR_BGR2HSV))
6
7 #Displaying
8 fig, ax = plt.subplots(1, 3, figsize= (12, 4))
9
10 ax[0].imshow(H, cmap='grey')
11 ax[0].set_title('Hue')
12
13 ax[1].imshow(S, cmap='grey')
14 ax[1].set_title('Saturation')
15
16 ax[2].imshow(V, cmap='grey')
17 ax[2].set_title('Value')
18
19 for a in ax:
20     a.axis('off')
```

Hue



Saturation



Value



Q6) b) *Best plane is the saturation plane*

. *Because it is the only plane where Jennifer is mostly bright compared to the background.*

```
1 #Because of the dark backgrounds selecting the threshold as 15 (a darker value)
2 threshold = 15
3
4 #Extracting the mask
5 _, mask = cv.threshold(S, threshold, 255, cv.THRESH_BINARY)
6
7 #Displaying the mask
8 fig, ax = plt.subplots(1, 1, figsize=(5, 5))
9 ax.imshow(mask, cmap='gray')
10 ax.set_title('Foreground Mask')
11 ax.axis('off')
12 plt.show()
```

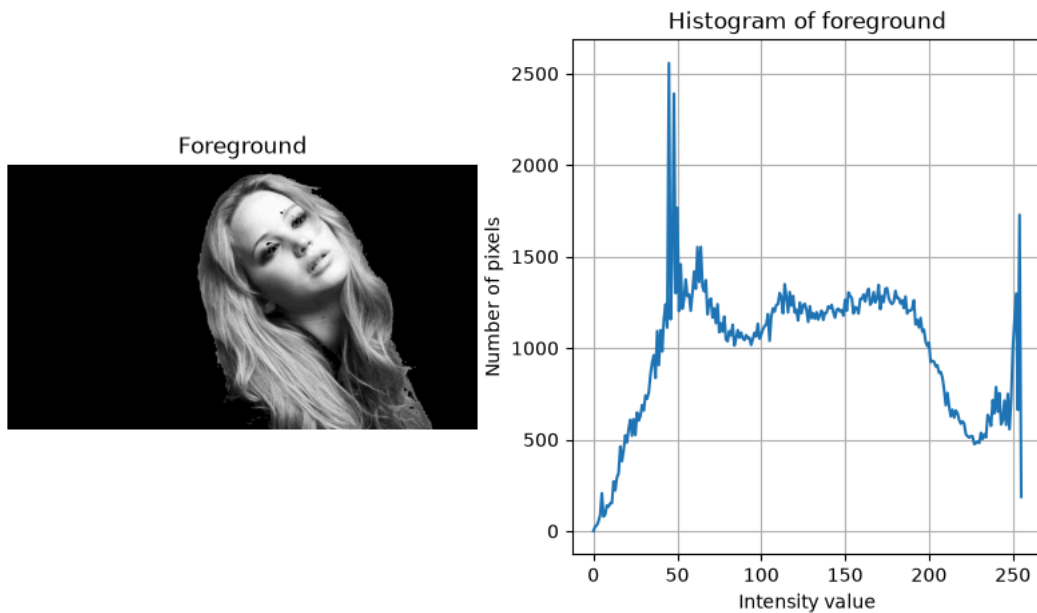
Foreground Mask



The saturation plane was selected because the foreground is mostly brighter than the background. Therefore, thresholding this plane gives a suitable foreground mask.

Q6) c)

```
1 #Obtaining foreground
2 foreground = cv.bitwise_and(V, V, mask=mask)
3
4 #Calculating histogram
5 foreground_pixels = V[mask == 255]
6 foreground_hist, foreground_bins = np.histogram(foreground_pixels, 256, [0,256])
7
8 #Displaying the mask
9 fig, ax = plt.subplots(1, 2, figsize=(10, 5))
10 ax[0].imshow(foreground, cmap='gray')
11 ax[0].set_title('Foreground')
12 ax[0].axis('off')
13
14 ax[1].plot(foreground_hist)
15 ax[1].set_title('Histogram of foreground')
16 ax[1].set_xlabel('Intensity value')
17 ax[1].set_ylabel('Number of pixels')
18 ax[1].grid()
19
20 plt.show()
21
```

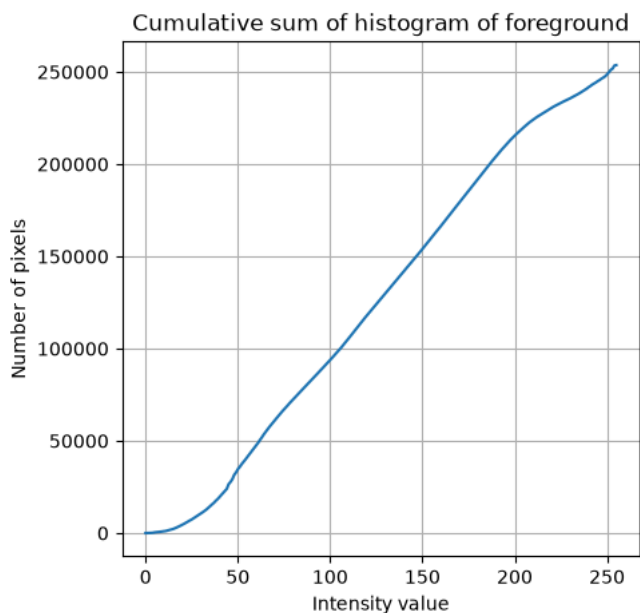


Q6) d)

```

1 #Obtaining the cumulative sum of the histogram
2 cdf = np.cumsum(foreground_hist)
3
4 fig, ax = plt.subplots(1, 1, figsize=(5,5))
5
6 ax.plot(cdf)
7 ax.set_title('Cumulative sum of histogram of foreground')
8 ax.set_xlabel('Intensity value')
9 ax.set_ylabel('Number of pixels')
10 ax.grid()
11
12 plt.show()
13

```



Q6) e)

```

1 #Equation from slides
2 L = 256
3 foreground_size = foreground_pixels.size
4 t = np.array([(L-1)/(foreground_size)*cdf[k] for k in range(256)])
5
6 t = np.round(t).astype(np.uint8)
7
8 #Applying equalization for the foreground
9 hist_eqd_foreground = np.zeros_like(V)

```

```
10 hist_eqd_foreground[mask == 255] = t[V[mask == 255]]
11
```

Q6) f)

```
1 #Extracting the background
2 background = cv.bitwise_and(V, V, mask=cv.bitwise_not(mask))
3
4 #Combining the background with histogram equalized foreground
5 final_V = cv.add(background, hist_eqd_foreground)
6
7 #Recombining H S V planes
8 final_HSV = cv.merge([H, S, final_V])
9
10 #Converting to RGB for display
11 final_BGR = cv.cvtColor(final_HSV, cv.COLOR_HSV2BGR)
12 final_RGB = cv.cvtColor(final_BGR, cv.COLOR_BGR2RGB)
13
14 #Displaying the mask
15 fig, ax = plt.subplots(1, 2, figsize=(10, 5))
16
17 ax[0].imshow(cv.cvtColor(im_6, cv.COLOR_BGR2RGB))
18 ax[0].set_title('Original image')
19 ax[0].axis('off')
20
21 ax[1].imshow(final_RGB)
22 ax[1].set_title('Final image')
23 ax[1].axis('off')
24 plt.show()
```

Original image



Final image



Histogram equalization was applied only to the foreground using the binary mask. The foreground contrast is enhanced while the background remains unchanged.

Q7) a)

```
1 #Importing image
2 im_7 = cv.imread('Albert.png', cv.IMREAD_GRAYSCALE)
3
4 #Sobel filter kernel
5 sobel = np.array([[1, 0, -1], [2, 0, -2], [1, 0, -1]])
6
7 #Existing filter2D function
8 im_7_part_a = cv.filter2D(im_7, cv.CV_64F, sobel)
```

Q7) b)

```
1 #Implementing from the scratch
2 #Creating an empty output image
3 im_7_part_b = np.zeros_like(im_7, dtype=np.float64)
4
5 #Getting row and column number
6 rows, cols = im_7.shape
7
8 #Going over the image
9 for i in range(1, rows - 1):
10     for j in range(1, cols - 1):
11
12         #Extracting the 3 by 3 neighborhood
13         region = im_7[i-1:i+2, j-1:j+2]
14
15         #Element wise multiplication and summation
16         im_7_part_b[i, j] = np.sum(region * sobel)
```

Q7) c)

```

1 #Using the outer product to create the kernel
2 vertical = np.array([1, 2, 1], dtype=np.float64)
3 horizontal = np.array([1, 0, -1], dtype=np.float64)
4
5 #Applying the horizontal 1D filter
6 temp = cv.filter2D(im_7, cv.CV_64F, horizontal.reshape(1, 3))
7
8 #Applying the vertical 1D filter
9 im_7_part_c = cv.filter2D(temp, cv.CV_64F, vertical.reshape(3, 1))

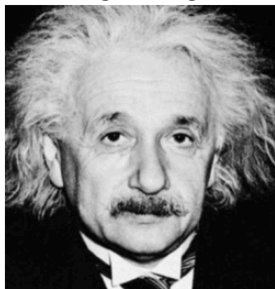
```

```

1 #Displaying the pictures
2 fig, ax = plt.subplots(1, 4, figsize=(12, 3))
3
4 ax[0].imshow(im_7, cmap='grey')
5 ax[0].set_title('Original image')
6
7 ax[1].imshow(np.abs(im_7_part_a), cmap='grey')
8 ax[1].set_title('Filter using filter2D')
9
10 ax[2].imshow(np.abs(im_7_part_b), cmap='grey')
11 ax[2].set_title('Filter from scratch')
12
13 ax[3].imshow(np.abs(im_7_part_c), cmap='grey')
14 ax[3].set_title('Filter using the property')
15
16 for a in ax:
17     a.axis('off')
18
19 plt.tight_layout()
20 plt.show()
21

```

Original image



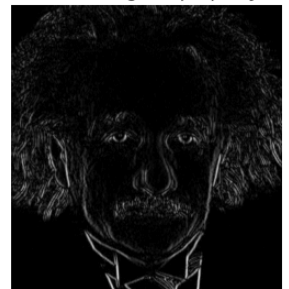
Filter using filter2D



Filter from scratch



Filter using the property



The Sobel filter detects changes in image intensity and therefore highlights the edges.
The built-in, manual, and separable implementations produce similar edge information.

Q8)

```

1 #Defining the function
2 def zoom_image(image, scale, method='nearest'):
3
4     #Getting the height and width
5     h, w = image.shape[:2]
6
7     #New height and width after scaling
8     new_h = int(h * scale)
9     new_w = int(w * scale)
10
11     #Creating a blank canvas to store pixel values after zooming
12     zoomed = np.zeros((new_h, new_w, image.shape[2]), dtype=image.dtype)
13
14     if method == 'nearest':
15
16         for i in range(new_h):
17             for j in range(new_w):
18
19                 #Inverse mapping
20                 src_y = i / scale
21                 src_x = j / scale
22

```

```

23         #Picking the nearest value
24         nearest_y = int(round(src_y))
25         nearest_x = int(round(src_x))
26
27         #Avoiding errors by going outside the image
28         nearest_y = min(nearest_y, h - 1)
29         nearest_x = min(nearest_x, w - 1)
30
31         #Assigning
32         zoomed[i, j] = image[nearest_y, nearest_x]
33
34     elif method == 'bilinear':
35
36         for i in range(new_h):
37             for j in range(new_w):
38
39                 #Inverse mapping
40                 src_y = i / scale
41                 src_x = j / scale
42
43                 #Finding the four pixels around
44                 y1 = int(np.floor(src_y))
45                 x1 = int(np.floor(src_x))
46
47                 y2 = min(y1 + 1, h - 1)
48                 x2 = min(x1 + 1, w - 1)
49
50                 #Measuring the distance of the zoomed pixel from the found four pixels
51                 dy = src_y - y1
52                 dx = src_x - x1
53
54                 #Linear interpolation
55                 top = (1 - dx) * image[y1, x1] + dx * image[y1, x2]
56                 bottom = (1 - dx) * image[y2, x1] + dx * image[y2, x2]
57
58                 value = (1 - dy) * top + dy * bottom
59
60                 #Assigning
61                 zoomed[i, j] = np.clip(value, 0, 255)
62
63     return zoomed
64
65 #Importing the small image
66 im_8 = cv.imread('Taylor_zoomed_out.jpg')
67
68 #Importing the large image
69 im_8_zoomed = cv.imread('Taylor_zoomed.jpg')
70
71 #Zooming scale
72 s = 4
73
74 #(a) nearest-neighbor zoom
75 im_8_nearest_neighbor = zoom_image(im_8, s, 'nearest')
76
77 #(b) bilinear interpolation
78 im_8_bilinear_interpolation = zoom_image(im_8, s, 'bilinear')
79
80 # Function for calculating the normalized SSD
81 def normalized_ssd(original, reconstructed):
82
83     #Converting to float to avoid overflow during subtraction and squaring
84     original = original.astype(np.float64)
85     reconstructed = reconstructed.astype(np.float64)
86
87     #Calculating the sum of squared differences
88     ssd = np.sum((original - reconstructed) ** 2)
89
90     #Normalizing using the energy of the original image
91     normalized_ssd = ssd / np.sum(original ** 2)
92
93     return normalized_ssd
94
95 #Checking whether the image sizes are equal
96 print("Original large image shape:", im_8_zoomed.shape)
97 print("Nearest-neighbor shape:", im_8_nearest_neighbor.shape)
98 print("Bilinear interpolation shape:", im_8_bilinear_interpolation.shape)
99
100 # Calculating normalized SSD
101 nearest_ssd = normalized_ssd(im_8_zoomed, im_8_nearest_neighbor)
102 bilinear_ssd = normalized_ssd(im_8_zoomed, im_8_bilinear_interpolation)
103
104 # Displaying the results

```

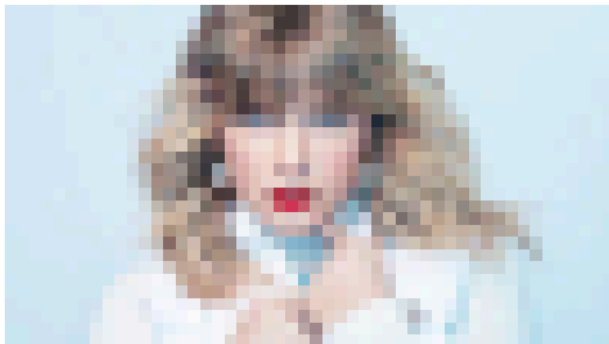
```

105 print("Normalized SSD for nearest-neighbor:", nearest_ssd)
106 print("Normalized SSD for bilinear interpolation:", bilinear_ssd)
107
108 #Displaying
109 fig, ax = plt.subplots(2, 2, figsize=(10, 6))
110
111 ax[0, 0].imshow(cv.cvtColor(im_8, cv.COLOR_BGR2RGB))
112 ax[0, 0].set_title('Original zoomed out image')
113
114 ax[0, 1].imshow(cv.cvtColor(im_8_zoomed, cv.COLOR_BGR2RGB))
115 ax[0, 1].set_title('Original zoomed image')
116
117 ax[1, 0].imshow(cv.cvtColor(im_8_nearest_neighbor, cv.COLOR_BGR2RGB))
118 ax[1, 0].set_title('Zoomed with nearest-neighbor')
119
120 ax[1, 1].imshow(cv.cvtColor(im_8_bilinear_interpolation, cv.COLOR_BGR2RGB))
121 ax[1, 1].set_title('Zoomed with bilinear interpolation')
122
123 for a in ax.flat:
124     a.axis('off')
125
126 plt.tight_layout()
127 plt.show()

```

Original large image shape: (112, 200, 3)
 Nearest-neighbor shape: (112, 200, 3)
 Bilinear interpolation shape: (112, 200, 3)
 Normalized SSD for nearest-neighbor: 0.009599429217788839
 Normalized SSD for bilinear interpolation: 0.007710873190589466

Original zoomed out image



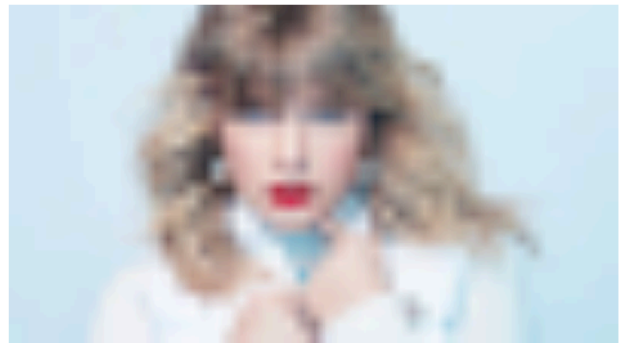
Zoomed with nearest-neighbor



Original zoomed image



Zoomed with bilinear interpolation



Nearest-neighbor interpolation selects the closest pixel, while bilinear interpolation uses the surrounding four pixels. The method with the lower normalized SSD gives the closer reconstruction of the original image.

Q9) a)

```

1 #Importing image
2 im_9 = cv.imread('Flower.jpg')
3
4 #Creating a blank mask
5 mask = np.zeros(im_9.shape[:2], np.uint8)
6
7 #For the use of grabCut algorithm
8 bgdModel = np.zeros((1, 65), np.float64)
9 fgdModel = np.zeros((1, 65), np.float64)
10
11 #Selecting the rectangle
12 rect = (60, 60, 600, 550)
13
14 #grabCut

```

```

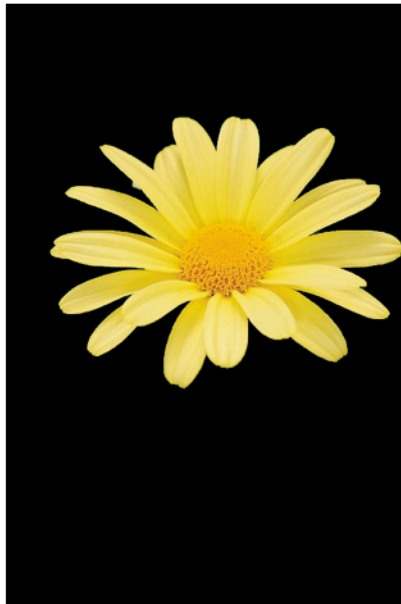
15 cv.grabCut(im_9, mask, rect, bgdModel, fgdModel, 5, cv.GC_INIT_WITH_RECT)
16
17 final_mask = np.where((mask == cv.GC_BGD) | (mask == cv.GC_PR_BGD), 0, 1).astype('uint8')
18
19 #Applying to the colour image
20 foreground = im_9 * final_mask[:, :, np.newaxis]
21
22 #For background
23 background_mask = 1 - final_mask
24
25 #Applying to the colour
26 background = im_9 * background_mask[:, :, np.newaxis]
27
28 #Display
29 fig, ax = plt.subplots(1, 3, figsize=(10, 5))
30
31 ax[0].imshow(final_mask, cmap='gray')
32 ax[0].set_title('Final segmentation mask')
33 ax[0].axis('off')
34
35 ax[1].imshow(cv.cvtColor(foreground, cv.COLOR_BGR2RGB))
36 ax[1].set_title('Foreground')
37 ax[1].axis('off')
38
39 ax[2].imshow(cv.cvtColor(background, cv.COLOR_BGR2RGB))
40 ax[2].set_title('Background')
41 ax[2].axis('off')
42
43 plt.tight_layout()
44 plt.show()

```

Final segmentation mask



Foreground



Background



GrabCut separates the flower from the background using foreground and background information. The resulting mask was used to extract the foreground and background separately.

Q9) b)

```

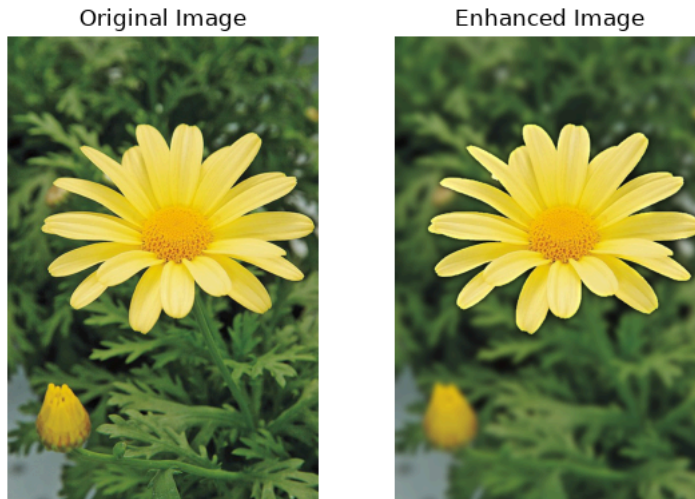
1 # Blurring the extracted background
2 blurred_background = cv.GaussianBlur(background, (31, 31), 0)
3
4 #Combining them
5 im_9_enhanced = cv.add(foreground, blurred_background)
6
7 #Displaying
8 fig, ax = plt.subplots(1, 2, figsize=(6, 4))
9
10 ax[0].imshow(cv.cvtColor(im_9, cv.COLOR_BGR2RGB))
11 ax[0].set_title('Original Image')
12 ax[0].axis('off')
13
14 ax[1].imshow(cv.cvtColor(im_9_enhanced, cv.COLOR_BGR2RGB))
15 ax[1].set_title('Enhanced Image')
16 ax[1].axis('off')

```

```

17
18 plt.tight_layout()
19 plt.show()
20

```



The background was blurred while the segmented flower was kept sharp. This makes the flower more prominent and creates a shallow depth-of-field effect.

Q9) c) The dark region appears because pixels outside the segmented foreground are set to zero. During blurring, these zero-valued pixels contribute to the averaging near the flower boundary, reducing the intensity of nearby background pixels and producing a dark halo.

Q10) a)

```

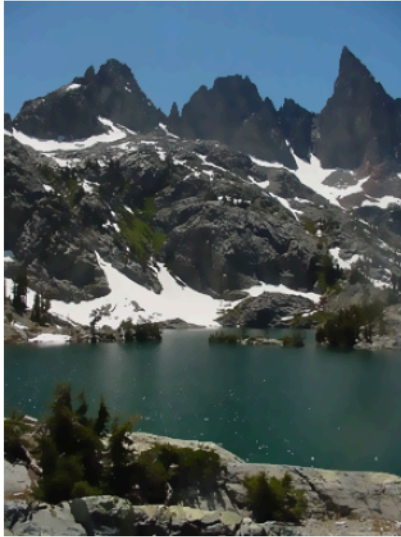
1 #Importing the image
2 im_10 = cv.imread('Lake.png')
3
4 #bilateral filter from opencv
5 sigma_s = 31
6 sigma_r = 41
7 kernal_size = 9
8 im_10_bilateral_builtin = cv.bilateralFilter(im_10, kernal_size, sigma_r, sigma_s)
9
10 #Gaussian filtering from openCV
11 im_10_gaussian_builtin = cv.GaussianBlur(im_10, (sigma_s, sigma_s), 0)
12
13 #Display
14 fig, ax = plt.subplots(1, 3, figsize=(10, 5))
15
16 ax[0].imshow(cv.cvtColor(im_10, cv.COLOR_BGR2RGB))
17 ax[0].set_title('Original image')
18 ax[0].axis('off')
19
20 ax[1].imshow(cv.cvtColor(im_10_bilateral_builtin, cv.COLOR_BGR2RGB))
21 ax[1].set_title('Filtered using builtin bilateralFilter')
22 ax[1].axis('off')
23
24 ax[2].imshow(cv.cvtColor(im_10_gaussian_builtin, cv.COLOR_BGR2RGB))
25 ax[2].set_title('Filtered using builtin GaussianBlur')
26 ax[2].axis('off')
27
28 plt.tight_layout()
29 plt.show()

```

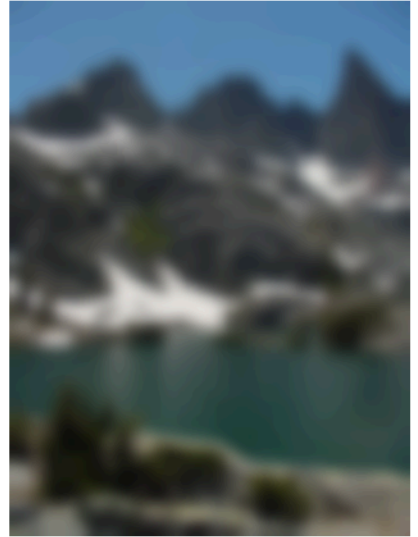
Original image



Filtered using builtin bilateralFilter



Filtered using builtin GaussianBlur



The Gaussian filter smooths the image but also blurs the edges.

The bilateral filter smooths the image while preserving the important edges more clearly.

```

1 '''
2
3
4
5
6
7
8
9
10
11
12
13
14
15
16
17
18
19
20
21
22
23
24 '''

```

Q10) b)

```

1 #Defining the function for bilateral filtering
2 def bilateral_filter(image, kernel_size, sigma_s, sigma_r):
3
4     #Getting the height and width of the image
5     h, w = image.shape[:2]
6
7     #Converting the image to float to avoid overflow during calculations
8     image_float = image.astype(np.float64)
9
10    #Creating a blank image to store the filtered result
11    filtered = np.zeros_like(image_float)
12
13    #Getting the radius of the kernel
14    radius = kernel_size // 2
15
16    #Padding the image to handle border pixels
17    padded = cv.copyMakeBorder(
18        image_float, radius, radius, radius, radius, cv.BORDER_REFLECT)
19
20    #Going through every pixel in the image
21    for i in range(h):
22        for j in range(w):
23

```

```

24         #Getting the center pixel
25         center_pixel = padded[i + radius, j + radius]
26
27         #Variables for weighted sum and total weight
28         weighted_sum = np.zeros(3, dtype=np.float64)
29
30         total_weight = 0.0
31
32         #Going through the neighborhood
33         for m in range(-radius, radius + 1):
34             for n in range(-radius, radius + 1):
35
36                 #Getting the neighboring pixel
37                 neighbor = padded[i + radius + m, j + radius + n]
38
39                 #Spatial distance between the center pixel
40                 #and the neighboring pixel
41                 spatial_distance = m**2 + n**2
42
43                 #Spatial Gaussian weight
44                 spatial_weight = np.exp(-spatial_distance / (2 * sigma_s**2))
45
46                 #Difference in intensity/color between
47                 #the center and neighboring pixel
48                 intensity_distance = np.sum((neighbor - center_pixel)**2)
49
50                 #Range Gaussian weight
51                 range_weight = np.exp(-intensity_distance / (2 * sigma_r**2))
52
53                 #Final bilateral weight
54                 weight = spatial_weight * range_weight
55
56                 #Adding weighted neighboring pixel
57                 weighted_sum += weight * neighbor
58
59                 #Adding the weight
60                 total_weight += weight
61
62         #Normalizing by the total weight
63         filtered[i, j] = weighted_sum / total_weight
64
65         #Limiting the values to the valid image range
66         filtered = np.clip(filtered, 0, 255)
67
68         #Converting back to uint8
69         return filtered.astype(np.uint8)
70
71 #From the created funtion
72 im_10_bilateral_created = bilateral_filter(im_10, kernal_size, sigma_s, sigma_r)
73
74 #Comparison
75 def mse(image1, image2):
76
77     image1 = image1.astype(np.float64)
78     image2 = image2.astype(np.float64)
79
80     error = np.mean((image1 - image2) ** 2)
81
82     return error
83
84 mse_value = mse(im_10_bilateral_builtin, im_10_bilateral_created)
85
86 print("MSE between OpenCV and created bilateral filter:", mse_value)
87
88 #Display
89 fig, ax = plt.subplots(1, 4, figsize=(15, 6))
90
91 ax[0].imshow(cv.cvtColor(im_10, cv.COLOR_BGR2RGB))
92 ax[0].set_title('Original image')
93 ax[0].axis('off')
94
95 ax[1].imshow(cv.cvtColor(im_10_bilateral_builtin, cv.COLOR_BGR2RGB))
96 ax[1].set_title('Filtered using builtin bilateralFilter')
97 ax[1].axis('off')
98
99 ax[2].imshow(cv.cvtColor(im_10_bilateral_created, cv.COLOR_BGR2RGB))
100 ax[2].set_title('Filtered using created bilateralFilter')
101 ax[2].axis('off')
102
103 ax[3].imshow(cv.cvtColor(im_10_gaussian_builtin, cv.COLOR_BGR2RGB))
104 ax[3].set_title('Filtered using builtin GaussianBlur')
105 ax[3].axis('off')

```

106
107

MSE between OpenCV and created bilateral filter: 14.905589412126808



The custom bilateral filter gives a result similar to the OpenCV bilateral filter.
A small MSE indicates that the two filtered images are close to each other.

```
1 '''
2
3
4
5
6
7
8
9
10
11
12
13
14
15
16
17
18
19
20
21
22
23
24
25
26
27
28
29
30
31
32
33
34
35
36
37
38
39
40
```